AKADEMIA NAUK STOSOWANYCH
W NOWYM SĄCZU

Wydział Nauk Inżynieryjnych
Katedra Informatyki

DOKUMENTACJA PROJEKTOWA

ZESPOŁOWE PRZEDSIĘWZIĘCIE INŻYNIERSKIE

SEE Trainer -
Asystent treningowy

Autor:
Adrian Gargula
Jakub Hajduk
Michał Bernardy

Prowadzący:
mgr inż. Nikodem Bulanda

Nowy Sącz 2026

Spis treści

1. Analiza projektu **4**

1.1. Kontekst problemu 4

1.2. Nazwa projektu 4

1.3. Wykorzystanie sztucznej inteligencji 5

1.3.1. Przygotowanie danych treningowych (offline) 5

1.3.2. Analiza techniki w aplikacji mobilnej (real-time) 6

1.3.3. Informacja zwrotna dla użytkownika 6

1.3.4. Dlaczego zastosowano AI 6

1.4. Cel projektu 7

1.5. Mierzalne kryteria sukcesu 7

2. Zakres **8**

2.1. Analiza wymagań 8

2.2. Wymagania funkcjonalne 8

2.3. Wymagania niefunkcjonalne 10

2.4. Wymagania integracyjne 11

3. Diagram przypadków użycia, scenariusze oraz diagram klas **12**

3.1. wyjaśnienie architektury 13

3.2. Scenariusze 15

3.3. Wykorzystane technologie 23

3.3.1. Język programowania i środowisko AI: Python 23

3.3.2. Framework uczenia głębokiego: PyTorch 24

3.3.3. Biblioteka obliczeń numerycznych: NumPy 25

3.3.4. Biblioteka przetwarzania obrazu: OpenCV 26

3.3.5. Format wymiany modeli: ONNX 27

3.3.6. Framework AI mobilnego: TensorFlow Lite 28

3.3.7. Środowisko developerskie: Android Studio 29

3.3.8. Język programowania: Kotlin 30

3.3.9. Rozpoznawanie mowy: Vosk 31

3.3.10. Synteza mowy: Android TextToSpeech 32

3.3.11. Obsługa kamery: CameraX 33
3.3.12. Baza danych lokalna: Room (SQLite) 34
3.3.13. Framework backendowy: FastAPI 35
3.3.14. Baza danych serwera: PostgreSQL 36
3.3.15. System kontroli wersji: GitHub 37

4. Implementacja **38**

5. Testowanie **39**

6. Analiza projektu **40**

6.1. Wymagania początkowe 40

7. Podsumowanie projektu **41**

Literatura **42**

Spis rysunków **43**

Spis tabel **44**

Spis listingów **45**

1. Analiza projektu

1.1. Kontekst problemu

Wykonywanie ćwiczeń fizycznych bez nadzoru specjalisty wiąże się z wysokim ryzykiem utrwalania błędnych wzorców ruchowych, co prowadzi do:

- przeciążeń stawów,
- zmniejszonej efektywności treningu,
- zwiększonego ryzyka kontuzji,
- nie osiągnięcia założonych celów,

Jednocześnie dynamiczny rozwój technologii mobilnych metod uczenia maszynowego (Computer Vision, Deep Learning) umożliwia implementację systemów analizy ruchu w czasie rzeczywistym na urządzeniach konsumenckich.

Projekt stanowi odpowiedź na potrzebę stworzenia systemu umożliwiającego automatyczną ocenę techniki ćwiczeń bez konieczności udziału trenera.

1.2. Nazwa projektu

SEE Trainer to nazwa naszej aplikacji mobilnej, która powstała w ramach projektu mającego na celu stworzenie systemu wspierającego poprawne wykonywanie ćwiczeń fizycznych w czasie rzeczywistym. Aplikacja wykorzystuje technologie uczenia maszynowego i estymacji pozy ciała (pose detection), aby analizować technikę użytkownika i udzielać natychmiastowej informacji zwrotnej w formie wizualnej i głosowej.

Nazwa SEE Trainer jest akronimem od trzech kluczowych kroków korzystania z aplikacji:

- S – Set – użytkownik ustawia telefon w odpowiedniej pozycji przed rozpoczęciem ćwiczenia.
- E – Execute – użytkownik wykonuje ćwiczenie, a system rejestruje ruch i analizuje poprawność techniki.
- E – Enjoy – użytkownik otrzymuje natychmiastową informację zwrotną i może cieszyć się efektywnym i bezpiecznym treningiem.

Ponadto, skrót SEE nawiązuje również do funkcji aplikacji związanej z analizą wizualną – system „widzi” ruch użytkownika i ocenia jego technikę, co stanowi dodatkowy kontekst znaczeniowy.

Wybór nazwy był podyktowany chęcią stworzenia krótkiego, łatwego do zapamiętania i profesjonalnie brzmiącego tytułu, który jednocześnie odzwierciedla funkcjonalność aplikacji i proces korzystania z niej. Nazwa jest uniwersalna i czytelna zarówno w języku polskim, jak i angielskim, co jest istotne w kontekście prezentacji projektu i potencjalnej publikacji wyników.

1.3. Wykorzystanie sztucznej inteligencji

W projekcie SEE Trainer sztuczna inteligencja pełni bardzo ważną rolę w analizie techniki wykonywania ćwiczeń w czasie rzeczywistym. System wykorzystuje połączenie detekcji punktów ciała oraz modeli głębokiego uczenia (Temporal CNN i Technique Classifier) do oceny poprawności ruchu oraz generowania informacji zwrotnej dla użytkownika.

1.3.1. Przygotowanie danych treningowych (offline)

Do przygotowania danych treningowych zastosowano MediaPipe Pose.

- Cel: Ekstrakcja punktów charakterystycznych ciała (landmarków) z wideo treningowego, które posłużą do stworzenia datasetu.
- Działanie: MediaPipe analizuje nagrania offline, wyznacza położenie punktów ciała i zapisuje sekwencje ruchu wraz z oznaczeniem poprawności wykonania ćwiczenia.
- Zalety:
  1. Przyspiesza tworzenie datasetu dla modeli AI
  2. Pozwala na automatyczne oznaczanie punktów ciała bez ręcznego tagowania

W tym etapie MediaPipe nie działa w aplikacji mobilnej w czasie rzeczywistym – służy jedynie do wygenerowania danych wejściowych dla modelu.

1.3.2. Analiza techniki w aplikacji mobilnej (real-time)

Na podstawie przygotowanego datasetu wytrenowano Temporal CNN i Technique Classifier, które działają w aplikacji mobilnej:

- Temporal CNN: analizuje sekwencje punktów ciała w czasie, wyodrębniając wzorce ruchu.
- Technique Classifier: klasyfikuje powtórzenie jako poprawne lub niepoprawne.

Modele te działają całkowicie offline na telefonie dzięki wykorzystaniu TensorFlow Lite, co zapewnia:

- pełną prywatność danych użytkownika,
- brak konieczności wysyłania obrazu do serwera,
- real-time feedback wizualny i głosowy.

1.3.3. Informacja zwrotna dla użytkownika

Na podstawie predykcji modeli AI aplikacja generuje:

- Tekstowy feedback – np. „Poprawnie” / „Błąd w kolanach”.
- Głosowy feedback – poprzez Android TextToSpeech API.
- Wizualizację punktów ciała i szkieletu – użytkownik może zobaczyć błędne elementy ruchu w czasie rzeczywistym.

Dzięki temu użytkownik natychmiast dowiaduje się, które aspekty techniki wymagają poprawy.

1.3.4. Dlaczego zastosowano AI

- Automatyczna analiza techniki bez potrzeby trenera.
- Działanie offline, w pełni lokalne na urządzeniu.
- Możliwość rozbudowy o kolejne ćwiczenia i modele.
- Natychmiastowy, intuicyjny feedback zwiększający bezpieczeństwo i efektywność treningu.

1.4. Cel projektu

Celem projektu jest zaprojektowanie i implementacja mobilnej aplikacji treningowej wykorzystującej model sztucznej inteligencji do analizy poprawności wykonywania ćwiczeń fizycznych w czasie rzeczywistym.

Projekt ma na celu stworzenie systemu wspomagającego użytkownika podczas treningu poprzez:

- wykrywanie pozycji ciała przy użyciu modelu typu pose estimation,
- analizę biomechaniczną ruchu na podstawie punktów kluczowych ciała,
- ocenę poprawności techniki wykonywanego ćwiczenia,
- generowanie natychmiastowej informacji zwrotnej w postaci komunikatu tekstowego lub głosowego,
- zliczanie powtórzeń oraz monitorowanie poprawnej techniki.

System będzie działał w oparciu o model sztucznej inteligencji przetwarzający obraz z kamery urządzenia mobilnego w czasie rzeczywistym. Analiza ruchu będzie oparta na współrzędnych punktów kluczowych ciała (np. barki, biodra, kolana, kostki), na podstawie których obliczane będą kąty w stawach oraz relacje przestrzenne pomiędzy segmentami ciała.

1.5. Mierzalne kryteria sukcesu

Projekt zostanie uznany za zrealizowany poprawnie, jeżeli:

- System osiągnie płynność minimum 15 FPS.
- Opóźnienie analizy nie przekroczy 300 ms.
- Dokładność zliczania powtórzeń wyniesie minimum 90
- System poprawnie rozpozna co najmniej 75 % błędów technicznych w testach eksperymentalnych.
- Aplikacja będzie stabilna przez minimum 30 minut ciągłej pracy.

2. Zakres

2.1. Analiza wymagań

Analiza wymagań została przeprowadzona w celu jednoznacznego określenia zakresu funkcjonalnego systemu, parametrów jakościowych oraz sposobu integracji komponentów. Wymagania zostały podzielone na trzy kategorie:

- wymagania funkcjonalne,
- wymagania niefunkcjonalne,
- wymagania integracyjne.

Każde wymaganie zostało opisane w sposób umożliwiający jego weryfikację podczas testów końcowych.

2.2. Wymagania funkcjonalne

Wymagania funkcjonalne określają, jakie operacje system musi realizować oraz jakie rezultaty musi generować.

Rejestracja obrazu

- System musi umożliwiać przechwytywanie obrazu z kamery urządzenia mobilnego.
- System musi przetwarzać obraz w czasie rzeczywistym.

Wykrywanie sylwetki użytkownika

- System musi wykrywać obecność użytkownika w kadrze.
- System musi identyfikować kluczowe punkty ciała dla danego ćwiczenia.
- System musi obsługiwać analizę jednej osoby w kadrze.

Analiza techniki ćwiczeń

- System musi analizować kąty w stawach (kolano, biodro, łokieć, bark).
- System musi wykrywać nieprawidłowości w technice ruchu.
- System musi oceniać zakres ruchu.
- System musi rozpoznawać rozpoczęcie i zakończenie powtórzenia.
- System musi zliczać powtórzenia.

Obsługa wielu ćwiczeń

- System musi umożliwiać wybór ćwiczenia z listy minimum 20 ćwiczeń.
- Dla każdego ćwiczenia system musi posiadać zdefiniowaną logikę analizy ruchu.

Informacja zwrotna

- System musi generować komunikaty tekstowe dotyczące poprawności wykonania ćwiczenia.
- System powinien umożliwiać generowanie komunikatów głosowych.
- System powinien wyświetlać komunikaty ostrzegawcze w przypadku błędnej techniki.

Zapisywanie danych

- System powinien zapisywać liczbę wykonanych powtórzeń.
- System powinien zapisywać procent poprawnie wykonanych powtórzeń (dla danego ćwiczenia).
- System powinien umożliwiać podgląd statystyk dla danego ćwiczenia.

2.3. Wymagania niefunkcjonalne

Wymagania niefunkcjonalne określają jakość działania systemu.

Wydajność

- System powinien działać z minimalną płynnością 15 klatek na sekundę.
- Opóźnienie analizy nie powinno przekraczać 300 ms.
- Model AI powinien być zoptymalizowany pod kątem urządzeń mobilnych, by nie przeciążać systemu.

Stabilność

- Aplikacja nie może powodować awarii systemu operacyjnego.
- System powinien poprawnie reagować na brak wykrytej osoby.

Użyteczność

- Czas rozpoczęcia analizy po wyborze ćwiczenia $\leq$ 2 sekundy.
- Interfejs musi być responsywny.
- System powinien zapewnić wysoką dokładność w odpowidziach dla użytkownika, aby uniknąć błędnych ruchów w ćwiczeniach.

Kompatybilność

- Aplikacja powinna działać na urządzeniach z systemem Android w wersji 8.0 lub nowszej.
- System powinien poprawnie działać na urządzeniach o różnych rozdzielczościach ekranów.
- Aplikacja będzie przeznaczona na urządzenia które obsługują AI w aparacie.

2.4. Wymagania integracyjne

Wymagania integracyjne określają sposób współpracy pomiędzy komponentami systemu.

Integracja z modelem AI

- Aplikacja musi być zintegrowana z modelem pose estimation.
- System musi przekazywać klatkę obrazu do modelu AI.
- Model musi zwracać współrzędne punktów kluczowych w ustandaryzowanym formacie.

Integracja modułu analizy z interfejsem

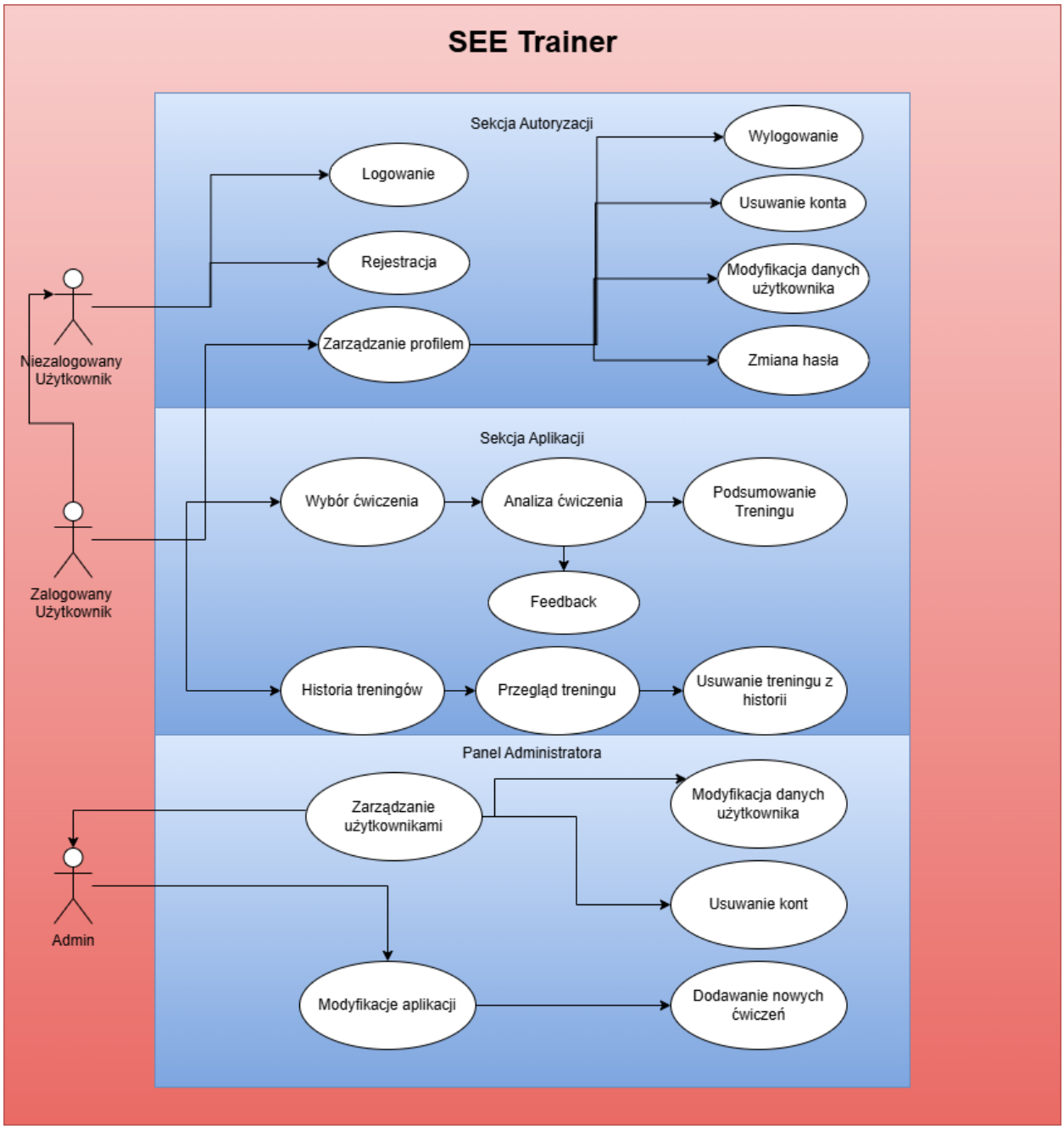
- Wyniki analizy muszą być przekazywane do warstwy prezentacji.
- Interfejs musi wyświetlać wynik analizy w czasie rzeczywistym.

Integracja z bazą danych

- Dane treningowe muszą być zapisywane po zakończeniu serii.
- System musi umożliwiać odczyt danych historycznych bez wpływu na wydajność analizy AI.

3. Diagram przypadków użycia, scenariusze oraz diagram klas

Diagram przypadków użycia dla aplikacji **SEE Trainer** znajduje się na rysunku nr 3.1 (s. 12):



Rys. 3.1. Diagram przypadków użycia aplikacji SEE Trainer

3.1. wyjaśnienie architektury

Warstwa użytkownika

- User
- Admin

Obsługa kont, logowania i zarządzania użytkownikami.

Warstwa treningowa

- Exercise
- TrainingSession
- TrainingHistory

Obsługa ćwiczeń, treningów i statystyk.

Warstwa AI

- CameraManager
- PoseEstimator
- TechniqueAnalyzer

Proces analizy ruchu: `Camera -> PoseEstimator -> TechniqueAnalyzer`

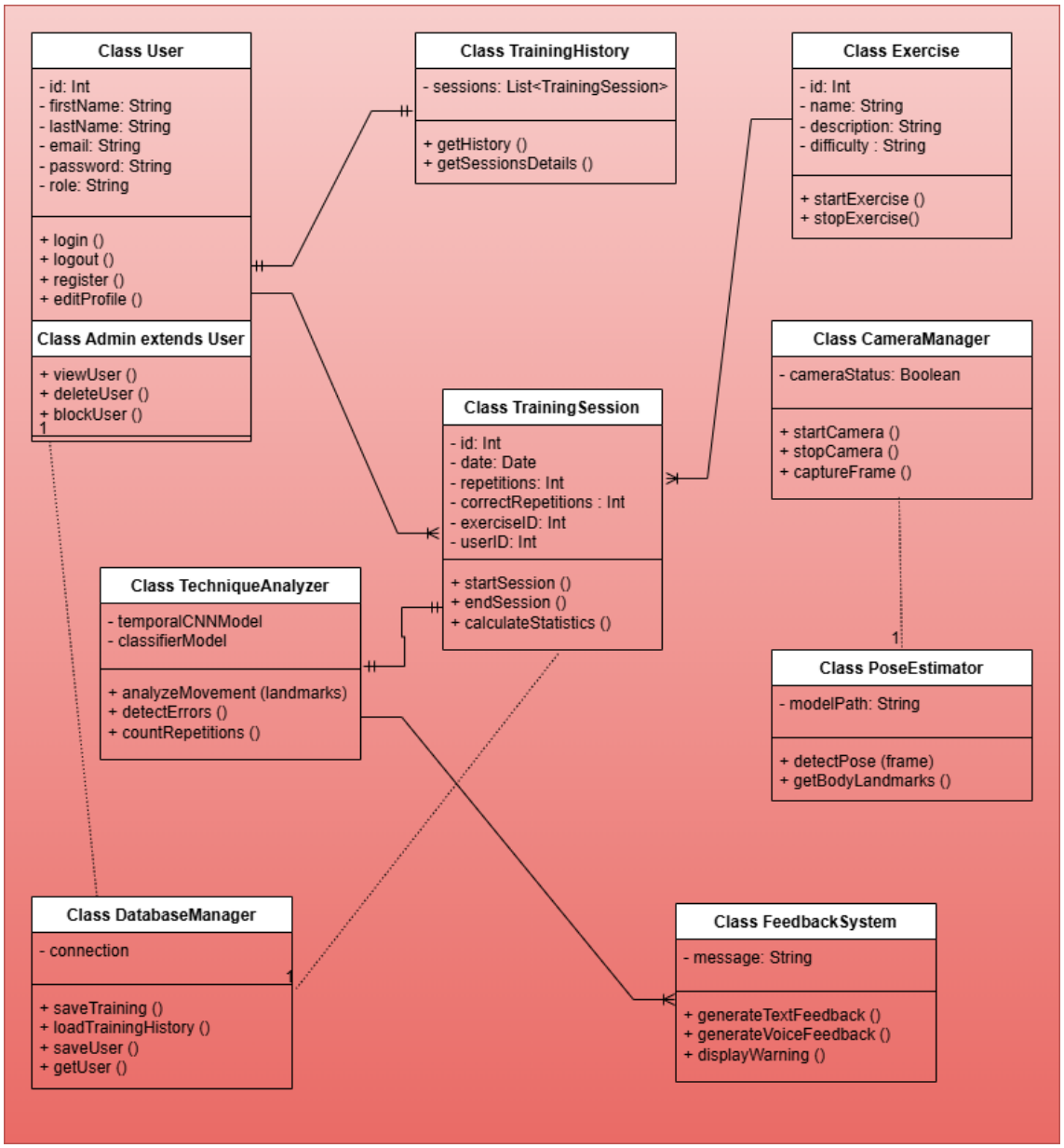
Warstwa komunikacji z użytkownikiem

- FeedbackSystem generuje:
  - komunikaty tekstowe
  - komunikaty głosowe
  - ostrzeżenia o błędach

Warstwa danych

- DatabaseManager zapisuje informacje o:
  - użytkownikach
  - treningach
  - historii ćwiczeń

Diagram klas aplikacji znajduje się na rysunku nr 3.2 (s. 14) poniżej:



Rys. 3.2. Diagram klas aplikacji SEE Trainer

3.2. Scenariusze

Scenariusze opisują sposób, w jaki użytkownicy wchodzą w interakcję z systemem, przedstawiając krok po kroku ich działania oraz oczekiwane rezultaty. Umożliwiają one dokładne zaplanowanie funkcjonowania platformy, zapewniają prawidłowy przebieg procesów i pozwalają zminimalizować ryzyko wystąpienia błędów. Stanowią również cenne wsparcie dla programistów i testerów, ponieważ pomagają im zrozumieć, jak system powinien reagować w różnych sytuacjach i scenariuszach użytkowania.

Tytuł	Logowanie do aplikacji
Numer	Scenariusz 1
Aktorzy	Użytkownik
Stan wejścia	Aktor uruchamia aplikację mobilną i widzi ekran logowania
Przebieg	1. Aktor wpisuje login i hasło 2. Aktor klika przycisk „Zaloguj się” 3. System weryfikuje dane w bazie użytkowników 4. Jeśli dane poprawne – użytkownik zostaje przeniesiony do ekranu głównego aplikacji
Wynik	Aktor zalogowany, dostęp do wszystkich funkcji aplikacji
Scenariusz alternatywny	4.a Aktor kliknął „Nie masz konta?” – zostaje przeniesiony do ekranu rejestracji 4.b Podane dane są niepoprawne – wyświetla się komunikat „Błędny login lub hasło”

Tab. 3.1. Scenariusz 1 - Logowanie do aplikacji

Tytuł	Rejestracja nowego użytkownika
Numer	Scenariusz 2
Aktorzy	Użytkownik
Stan wejścia	Aktor kliknął „Nie masz konta?” na ekranie logowania
Przebieg	1. Aktor wpisuje wymagane dane: imię, nazwisko, email, login, hasło 2. Aktor klika przycisk „Zarejestruj się” 3. System zapisuje dane w bazie użytkowników 4. Aktor zostaje automatycznie zalogowany i przeniesiony do ekranu głównego
Wynik	Konto użytkownika zostało utworzone, użytkownik zalogowany, dostęp do listy ćwiczeń i funkcji treningowych
Scenariusz alternatywny	3.a Podany email lub login już istnieje – aplikacja wyświetla komunikat „Użytkownik już istnieje” 4.a Aktor nie wypełnił wszystkich pól – wyświetlany jest komunikat „Uzupełnij wszystkie wymagane pola”.

Tab. 3.2. Scenariusz 2 - Rejestracja nowego użytkownika

Tytuł	Wykonanie ćwiczenia i analiza techniki
Numer	Scenariusz 3
Aktorzy	Użytkownik
Stan wejścia	Aktor jest zalogowany w aplikacji mobilnej i widzi listę dostępnych ćwiczeń
Przebieg	1. Aktor wybiera ćwiczenie (np. przysiad) 2. Aktor uruchamia kamerę w aplikacji 3. Model AI wykrywa sylwetkę i punkty kluczowe 4. Model analizuje ruch i technikę w czasie rzeczywistym 5. System wyświetla wizualizację punktów ciała i linie połączeń 6. System generuje feedback głosowy w przypadku błędnej techniki
Wynik	Aktor otrzymuje wizualną i głosową informację zwrotną na temat poprawności wykonania ćwiczenia
Scenariusz alternatywny	4.a Aktor przerywa ćwiczenie – analiza zostaje zatrzymana 4.b AI nie wykrywa sylwetki – aplikacja wyświetla komunikat „Nie wykryto użytkownika”

Tab. 3.3. Scenariusz 3 - Wykonanie ćwiczenia i analiza techniki

Tytuł	Historia treningów
Numer	Scenariusz 4
Aktorzy	Użytkownik
Stan wejścia	Aktor jest zalogowany w aplikacji mobilnej i może przeglądać historię swoich treningów
Przebieg	1. Użytkownik wchodzi w sekcję Historia. 2. Aplikacja wyświetla listę treningów. 3. Użytkownik może zobaczyć: (a) liczbę powtórzeń, (b) poprawność techniki, (c) datę treningu.
Wynik	Aktor otrzymuje dostęp do historii swoich treningów które może przeglądać
Scenariusz alternatywny	4.a Aktor robi pierwszy trening – historia treningów wyświetla się pusta.

Tab. 3.4. Scenariusz 4 - Historia treningów

Tytuł	Wylogowanie
Numer	Scenariusz 5
Aktorzy	Użytkownik
Stan wejścia	Aktor zalogowany do aplikacji mobilnej klikną wyloguj
Przebieg	1. Aktor naciska przycisk "wyloguj" 2. Aplikacja usuwa sesję logowania. 3. Użytkownik wraca do ekranu logowania.
Wynik	Zakończenie sesji użytkownika.
Scenariusz alternatywny	4.a Aktor przypadkiem naciska guzik wylogowania, system wyświetla komunikat "Czy napewno chcesz się wylogować?"

Tab. 3.5. Scenariusz 5 - Wylogowanie

Tytuł	Zarządzanie użytkownikami
Numer	Scenariusz 6
Aktorzy	Admin
Stan wejścia	Admin zalogowany do aplikacji mobilnej
Przebieg	1. Admin po zalogowaniu jest w stanie za pomocą panelu administratora: (a) przeglądać użytkowników, (b) usuwać konto, (c) blokować konto. 2. Admin wybiera opcje usunięcia konkretnego użytkownika, 3. Użytkownik zostaje usunięty,
Wynik	Usunięcie użytkownika.
Scenariusz alternatywny	4.a Błąd podczas usuwania użytkownika - "Brak połączenia"

Tab. 3.6. Scenariusz 6 - Zarządzanie użytkownikami

Tytuł	Edycja profilu
Numer	Scenariusz 7
Aktorzy	Użytkownik
Stan wejścia	Użytkownik zalogowany do aplikacji mobilnej klika "edycja profilu"
Przebieg	1. Aktor otwiera profil. (a) Aktor zmienia dane (np. email lub hasło). 2. Aktor zapisuje zmiany. 3. System aktualizuje dane w bazie.
Wynik	Modyfikacja danych użytkownika.
Scenariusz alternatywny	4.a Aktor zmienia dane na te same co poprzednie, wyświetla komunikat, że nie może zmienić na te same dane. 4.b Aktor zmienia dane które nie spełniają warunków walidacji, wyświetla komunikat "wprowadź poprawne dane".

Tab. 3.7. Scenariusz 7 - Edycja profilu

Tytuł	Zapis wyników treningu
Numer	Scenariusz 8
Aktorzy	Użytkownik
Stan wejścia	Użytkownik zakończył analizę ćwiczenia.
Przebieg	1. System kończy analizę ćwiczenia. 2. System oblicza statystyki treningu (liczba powtórzeń, poprawność techniki). 3. System zapisuje dane treningu w bazie danych. 4. Użytkownik otrzymuje podsumowanie treningu.
Wynik	Trening zostaje zapisany w historii użytkownika.
Scenariusz alternatywny	4.a Wystąpił błąd zapisu – system wyświetla komunikat „Nie udało się zapisać treningu”.

Tab. 3.8. Scenariusz 8 - Zapis wyników treningu

3.3. Wykorzystane technologie

3.3.1. Język programowania i środowisko AI: Python

Python[1]: Popularny język programowania wysokiego poziomu szeroko stosowany w dziedzinie sztucznej inteligencji, analizy danych oraz uczenia maszynowego. Dzięki czytelnej składni oraz bardzo bogatemu ekosystemowi bibliotek naukowych Python stał się jednym z najczęściej wykorzystywanych języków w projektach związanych z wizją komputerową i trenowaniem modeli AI.

W projekcie aplikacji mobilnej analizującej technikę ćwiczeń Python pełni rolę dosyć ważną rolę na etapie trenowania modeli sztucznej inteligencji.

W szczególności Python odpowiada za:

- implementację pipeline'u trenowania modeli wykrywających punkty ciała,
- przygotowanie oraz przetwarzanie danych treningowych,
- integrację z bibliotekami AI takimi jak PyTorch, NumPy czy OpenCV,
- eksperymentowanie z architekturami modeli.

Python został wybrany ze względu na ogromną popularność w środowisku sztucznej inteligencji, dużą liczbę dostępnych bibliotek oraz bardzo dobrą integrację z narzędziami do uczenia głębokiego.

Rys. 3.3. Język programowania i środowisko AI: Python[1]

3.3.2. Framework uczenia głębokiego: PyTorch

PyTorch[2]: Biblioteka open source służąca do budowania i trenowania modeli uczenia głębokiego, rozwijana przez Meta AI. PyTorch umożliwia tworzenie złożonych sieci neuronowych oraz ich efektywne trenowanie z wykorzystaniem procesorów graficznych.

W projekcie aplikacji PyTorch wykorzystywany jest do:

- trenowania modeli wykrywających punkty ciała użytkownika,
- implementacji architektury modeli pose estimation,
- trenowania modeli analizujących ruch użytkownika,
- optymalizacji parametrów sieci neuronowych.

PyTorch został wybrany ze względu na dużą elastyczność oraz oraz szerokie zastosowanie w projektach badawczych związanych z wizją komputerową.

Rys. 3.4. Framework uczenia głębokiego: PyTorch[2]

3.3.3. Biblioteka obliczeń numerycznych: NumPy

NumPy[3]: Biblioteka języka Python umożliwiająca wykonywanie wydajnych operacji matematycznych na wielowymiarowych tablicach danych.

W projekcie NumPy wykorzystywane jest do:

- przetwarzania danych wejściowych modeli AI,
- manipulacji współrzędnymi punktów ciała,
- wykonywania operacji matematycznych podczas analizy ruchu użytkownika,
- obliczania kątów pomiędzy segmentami ciała podczas ćwiczeń.

NumPy zapewnia wysoką wydajność operacji numerycznych i stanowi podstawę wielu innych bibliotek wykorzystywanych w projektach uczenia maszynowego.

Rys. 3.5. Biblioteka obliczeń numerycznych: NumPy[3]

3.3.4. Biblioteka przetwarzania obrazu: OpenCV

OpenCV[4]: Biblioteka open source przeznaczona do przetwarzania obrazu oraz zastosowań wizji komputerowej.

W projekcie OpenCV wykorzystywane jest do:

- przetwarzania obrazu z nagrań treningowych,
- przygotowywania danych wejściowych dla modeli sztucznej inteligencji,
- analizy obrazu przed przekazaniem go do sieci neuronowej,
- wizualizacji punktów ciała wykrywanych przez model.

OpenCV jest jedną z najpopularniejszych bibliotek do przetwarzania obrazu i zapewnia wysoką wydajność podczas pracy z materiałem wideo.

Rys. 3.6. Biblioteka przetwarzania obrazu: OpenCV[4]

3.3.5. Format wymiany modeli: ONNX

ONNX[5]: Open Neural Network Exchange (ONNX) to otwarty format plików przeznaczony do przenoszenia modeli sztucznej inteligencji między różnymi frameworkami uczenia maszynowego, takimi jak PyTorch, TensorFlow czy Caffe. ONNX pozwala trenować model w jednym środowisku i uruchamiać go w innym, dzięki czemu bardzo ułatwia integrację modeli z różnymi systemami.

W projekcie ONNX wykorzystywane jest do:

- eksportowania wytrenowanych modeli Pose Estimation Model, Temporal CNN i Technique Classifier z PyTorch do formatu kompatybilnego z TensorFlow i TensorFlow Lite,
- zapewnienia kompatybilności modeli pomiędzy środowiskiem treningowym (komputer z GPU) a aplikacją mobilną na Androida,
- optymalizacji modeli przed wdrożeniem na urządzenia mobilne, w tym redukcji rozmiaru pliku i zmiany precyzji obliczeń (np. z float32 na float16),
- wspierania iteracyjnego procesu trenowania i wdrażania modeli dzięki łatwemu eksportowi i importowi plików ONNX.

ONNX został wybrany ze względu na swoją uniwersalność, możliwość łatwego przenoszenia modeli między frameworkami oraz zapewnienie spójności działania modeli w różnych środowiskach.

Rys. 3.7. Format wymiany modeli: ONNX[5]

3.3.6. Framework AI mobilnego: TensorFlow Lite

TensorFlow Lite[6]: Lekka wersja frameworka TensorFlow stworzona z myślą o urządzeniach mobilnych, umożliwiająca uruchamianie modeli sztucznej inteligencji w czasie rzeczywistym bez konieczności wysyłania danych na serwer.

W projekcie TensorFlow Lite wykorzystywane jest do:

- uruchamiania modeli Pose Estimation Model, Temporal CNN i Technique Classifier bezpośrednio na telefonie,
- przetwarzania obrazów i sekwencji wideo z kamery w czasie rzeczywistym,
- minimalizacji zużycia pamięci i energii urządzenia,
- integracji z Android Studio i Kotlin w celu implementacji funkcjonalności AI w aplikacji mobilnej,
- zapewnienia płynnej i responsywnej pracy aplikacji bez opóźnień w wizualizacji wyników.

TensorFlow Lite został wybrany ze względu na wydajność, kompatybilność z Androidem oraz możliwość pracy w trybie offline, zwiększa to znacznie prywatność użytkownika i szybkość działania aplikacji.

Rys. 3.8. Framework AI mobilnego: TensorFlow Lite[6]

3.3.7. Środowisko developerskie: Android Studio

Android Studio[7]: Oficjalne środowisko IDE do tworzenia aplikacji na system Android, oferujące narzędzia do programowania, debugowania, testowania i projektowania interfejsu użytkownika.

W projekcie Android Studio wykorzystywane jest do:

- tworzenia aplikacji mobilnej analizującej technikę ćwiczeń,
- integracji z TensorFlow Lite, OpenCV Android i CameraX,
- implementacji interfejsu wizualizującego punkty ciała oraz informacje zwrotne dla użytkownika,
- testowania aplikacji na emulatorach i rzeczywistych urządzeniach Android,
- zarządzania strukturą projektu i bibliotekami niezbędnymi do pracy z AI.

Android Studio zostało wybrane ze względu na kompleksowość środowiska, dostęp do narzędzi debugowania i wsparcie dla języka Kotlin.

Rys. 3.9. Środowisko developerskie: Android Studio[7]

3.3.8. Język programowania: Kotlin

Kotlin[8]: Nowoczesny język programowania przeznaczony do tworzenia aplikacji Android, oferujący czytelną składnię, bezpieczeństwo typów i pełną interoperacyjność z Javą.

W projekcie Kotlin wykorzystywane jest do:

- implementacji logiki aplikacji mobilnej, w tym obsługi kamer, modeli AI i wizualizacji wyników,
- tworzenia modułów integrujących TensorFlow Lite, OpenCV Android i CameraX,
- zapewnienia wysokiej czytelności kodu i łatwej rozbudowy aplikacji w przyszłości,
- integracji z nowoczesnymi narzędziami Android i bibliotekami UI.

Kotlin został wybrany ze względu na prostotę kodowania, bezpieczeństwo typów i łatwą integrację z ekosystemem Android.

Rys. 3.10. Język programowania: Kotlin[8]

3.3.9. Rozpoznawanie mowy: Vosk

Vosk[9]: Open source'owy silnik Speech-to-Text umożliwiający rozpoznawanie mowy w czasie rzeczywistym bez konieczności przesyłania danych do chmury.

W projekcie Vosk wykorzystywane jest do:

- rozpoznawania poleceń głosowych użytkownika,
- aktywacji funkcji aplikacji i nawigacji głosowej,
- współpracy z interfejsem użytkownika w czasie rzeczywistym,
- poprawy interakcji użytkownika z aplikacją mobilną podczas treningu,
- umożliwienia pracy w trybie offline, bez potrzeby dostępu do internetu.

Vosk został wybrany ze względu na dokładność rozpoznawania mowy, pracę w czasie rzeczywistym oraz brak konieczności przesyłania danych na serwery zewnętrzne.

Rys. 3.11. Rozpoznawanie mowy: Vosk[9]

3.3.10. Synteza mowy: Android TextToSpeech

Android TextToSpeech[10]: Wbudowana w Android systemowa funkcja umożliwiająca generowanie dźwięku z tekstu, umożliwia to komunikację między aplikacja a użytkownikiem.

W projekcie TextToSpeech wykorzystywane jest do:

- odczytywania informacji zwrotnych dotyczących poprawności techniki ćwiczeń,
- komunikowania wskazówek i sugestii w czasie rzeczywistym,
- poprawy interakcji użytkownika z aplikacją podczas treningu,
- zwiększenia zaangażowania użytkownika poprzez komunikaty audio,
- integracji z innymi funkcjami aplikacji, np. wizualizacją punktów ciała i klasyfikacją ruchu.

Android TextToSpeech został wybrany ze względu na prostotę integracji, kompatybilność z każdym telefonem Android i możliwość pracy w czasie rzeczywistym.

3.3.11. Obsługa kamery: CameraX

CameraX[11]: Biblioteka Android umożliwiająca łatwe i elastyczne korzystanie z aparatu w aplikacjach mobilnych.

W projekcie CameraX wykorzystywane jest do:

- przechwytywania obrazu i wideo w czasie rzeczywistym,
- dostarczania klatek do Pose Estimation Model i Temporal CNN,
- obsługi różnych ustawień kamery, takich jak rozdzielczość i tryb autofokusa,
- integracji z OpenCV Android i TensorFlow Lite w celu przetwarzania obrazu,
- zapewnienia płynnej pracy aplikacji niezależnie od modelu telefonu.

CameraX został wybrany ze względu na prostotę użycia, elastyczność i kompatybilność z nowoczesnymi telefonami Android.

3.3.12. Baza danych lokalna: Room (SQLite)

Room (SQLite)[12]: biblioteka Androida stanowiąca warstwę abstrakcji nad bazą danych SQLite, która umożliwia łatwe i bezpieczne przechowywanie danych w aplikacji mobilnej. Ułatwia ona pracę z bazą danych poprzez wykorzystanie obiektów i zapytań SQL, jednocześnie zapewniając kontrolę typów oraz bezpieczeństwo operacji na danych.

W projekcie Room wykorzystywane jest do:

- przechowywania danych użytkownika w aplikacji mobilnej,
- zapisywania historii wykonanych treningów,
- przechowywania informacji o ćwiczeniach oraz ich parametrach,
- zapisywania wyników analizy techniki wykonywania ćwiczeń,
- umożliwienia działania aplikacji w trybie offline bez połączenia z internetem.

Room został wybrany ze względu na dobrą integrację z systemem Android, wysoką wydajność działania oraz uproszczenie pracy z bazą danych poprzez warstwę abstrakcji nad SQLite.

Rys. 3.12. Baza danych lokalna: Room (SQLite)[12]

3.3.13. Framework backendowy: FastAPI

FastAPI[13]: nowoczesny framework do tworzenia interfejsów API w języku Python, który umożliwia szybkie budowanie wydajnych aplikacji serwerowych. Charakteryzuje się wysoką wydajnością, prostotą implementacji oraz automatyczną dokumentacją API.

W projekcie FastAPI wykorzystywane jest do:

- obsługi komunikacji pomiędzy aplikacją mobilną a serwerem,
- zarządzania kontami użytkowników oraz procesem logowania i rejestracji,
- synchronizacji danych treningowych pomiędzy urządzeniem mobilnym a bazą danych,
- udostępniania interfejsu API do przesyłania oraz pobierania danych.

FastAPI zostało wybrane ze względu na wysoką wydajność, łatwą integrację z Pythonem oraz prostotę tworzenia nowoczesnych API REST.

Rys. 3.13. Framework backendowy: FastAPI[13]

3.3.14. Baza danych serwera: PostgreSQL

PostgreSQL[14]: zaawansowany relacyjny system zarządzania bazą danych typu open source, wykorzystywany do przechowywania i zarządzania dużą ilością danych w aplikacjach serwerowych. System ten zapewnia wysoką wydajność, bezpieczeństwo oraz obsługę zaawansowanych zapytań SQL.

W projekcie PostgreSQL wykorzystywany jest do:

- przechowywania danych użytkowników aplikacji,
- zapisywania historii treningów i statystyk użytkowników,
- przechowywania informacji o dostępnych ćwiczeniach,
- obsługi danych synchronizowanych z aplikacji mobilnej,
- zapewnienia trwałości oraz bezpieczeństwa przechowywanych danych.

PostgreSQL został wybrany ze względu na wysoką stabilność, dużą wydajność przy pracy z dużymi zbiorami danych oraz szerokie możliwości integracji z systemami backendowymi.

Rys. 3.14. Baza danych serwera: PostgreSQL[14]

3.3.15. System kontroli wersji: GitHub

GitHub[15]: platforma internetowa służąca do zarządzania kodem źródłowym oraz współpracy zespołowej nad projektami programistycznymi. Wykorzystuje system kontroli wersji Git, który umożliwia śledzenie zmian w kodzie, zarządzanie różnymi wersjami projektu oraz współpracę wielu programistów nad tym samym repozytorium.

W projekcie GitHub wykorzystywany jest do:

- przechowywania kodu źródłowego aplikacji mobilnej oraz komponentów systemu,
- śledzenia zmian wprowadzanych w projekcie przez członków zespołu,
- zarządzania wersjami oprogramowania,
- umożliwienia współpracy zespołowej poprzez system commitów, branchy oraz pull requestów,
- przechowywania historii rozwoju projektu oraz dokumentacji technicznej.

GitHub został wybrany ze względu na szeroką popularność wśród programistów, łatwość integracji z narzędziami programistycznymi oraz możliwość efektywnej współpracy zespołowej przy rozwijaniu projektów programistycznych.

Rys. 3.15. Platforma GitHub wykorzystywana w projekcie[15]

4. Implementacja

5. Testowanie

6. Analiza projektu

6.1. Wymagania początkowe

7. Podsumowanie projektu

Bibliografia

[1] *Strona internetowa Wikipedia*. URL: https://en.wikipedia.org/wiki/Python_(programming_language) (term. wiz. 08. 03. 2026).

[2] *Strona internetowa PyTorch*. URL: https://pytorch.org (term. wiz. 08. 03. 2026).

[3] *Strona internetowa NumPy*. URL: https://numpy.org (term. wiz. 08. 03. 2026).

[4] *Strona internetowa OpenCV*. URL: https://opencv.org (term. wiz. 08. 03. 2026).

[5] *Strona internetowa ONNX*. URL: https://onnx.ai (term. wiz. 08. 03. 2026).

[6] *Strona internetowa GitHub*. URL: https://github.com/tensorflow/tensorflow/tree/master/tensorflow/lite (term. wiz. 08. 03. 2026).

[7] *Strona internetowa Android Studio*. URL: https://developer.android.com/?hl=pl (term. wiz. 08. 03. 2026).

[8] *Strona internetowa Kotlin*. URL: https://kotlinlang.org (term. wiz. 08. 03. 2026).

[9] *Strona internetowa Alphacephei*. URL: https://alphacephei.com/vosk/ (term. wiz. 08. 03. 2026).

[10] *Strona internetowa Android Studio*. URL: https://developer.android.com/reference/android/speech/tts/TextToSpeech (term. wiz. 08. 03. 2026).

[11] *Strona internetowa Android Studio*. URL: https://developer.android.com/media/camera/camerax?hl=pl (term. wiz. 08. 03. 2026).

[12] *Strona internetowa Android Studio*. URL: https://developer.android.com/training/data-storage/room?hl=pl (term. wiz. 08. 04. 2026).

[13] *Strona internetowa FastAPI*. URL: https://fastapi.tiangolo.com/ (term. wiz. 08. 04. 2026).

[14] *Strona internetowa PostgreSQL*. URL: https://www.postgresql.org.pl/ (term. wiz. 08. 04. 2026).

[15] *Strona internetowa GitHub*. URL: https://github.com/ (term. wiz. 08. 04. 2026).

Spis rysunków

3.1. Diagram przypadków użycia aplikacji SEE Trainer 12
3.2. Diagram klas aplikacji SEE Trainer 14
3.3. Język programowania i środowisko AI: Python[1] 23
3.4. Framework uczenia głębokiego: PyTorch[2] 24
3.5. Biblioteka obliczeń numerycznych: NumPy[3] 25
3.6. Biblioteka przetwarzania obrazu: OpenCV[4] 26
3.7. Format wymiany modeli: ONNX[5] 27
3.8. Framework AI mobilnego: TensorFlow Lite[6] 28
3.9. Środowisko developerskie: Android Studio[7] 29
3.10. Język programowania: Kotlin[8] 30
3.11. Rozpoznawanie mowy: Vosk[9] 31
3.12. Baza danych lokalna: Room (SQLite)[12] 34
3.13. Framework backendowy: FastAPI[13] 35
3.14. Baza danych serwera: PostgreSQL[14] 36
3.15. Platforma GitHub wykorzystywana w projekcie[15] 37

Spis tabel

3.1. Scenariusz 1 - Logowanie do aplikacji 15
3.2. Scenariusz 2 - Rejestracja nowego użytkownika 16
3.3. Scenariusz 3 - Wykonanie ćwiczenia i analiza techniki 17
3.4. Scenariusz 4 - Historia treningów 18
3.5. Scenariusz 5 - Wylogowanie 19
3.6. Scenariusz 6 - Zarządzanie użytkownikami 20
3.7. Scenariusz 7 - Edycja profilu 21
3.8. Scenariusz 8 - Zapis wyników treningu 22

Spis listingów